

Can Kultur
Carnegie Mellon University
ckultur@cs.cmu.edu

Jaromir Savelka
Carnegie Mellon University
jsavelka@cs.cmu.edu

Christopher Bogart
Carnegie Mellon University
cbogart@cs.cmu.edu

Majd Sakr
Carnegie Mellon University
msakr@cs.cmu.edu

Context and Background of the Problem

CS1 courses are vital for STEM careers, forming the basis for advanced study and professional success. CS1 courses:

- teach **fundamental** concepts,
- emphasize **problem-solving**, and
- involve **hands-on** learning.

Programming concepts are **interdependent**:

- **understanding** one helps grasp others,
- **struggling** with one makes others harder to learn.

Project-Based Learning (PBL)

- boosts **engagement** and **effectiveness** in CS education
- by focusing on **real-world problems** and applications,
- which build student **interest** and **confidence**.

Effective learning requires **timely, contextualized feedback**, guiding students' progress and study habits.

The **review-feedback cycle** in IT/CS courses can be **lengthy**, as submissions are often hard to access and review while in process.

Automatic assessment of programming assignments is promising to **save time**, in terms of **grading** and **producing timely, actionable feedback**.

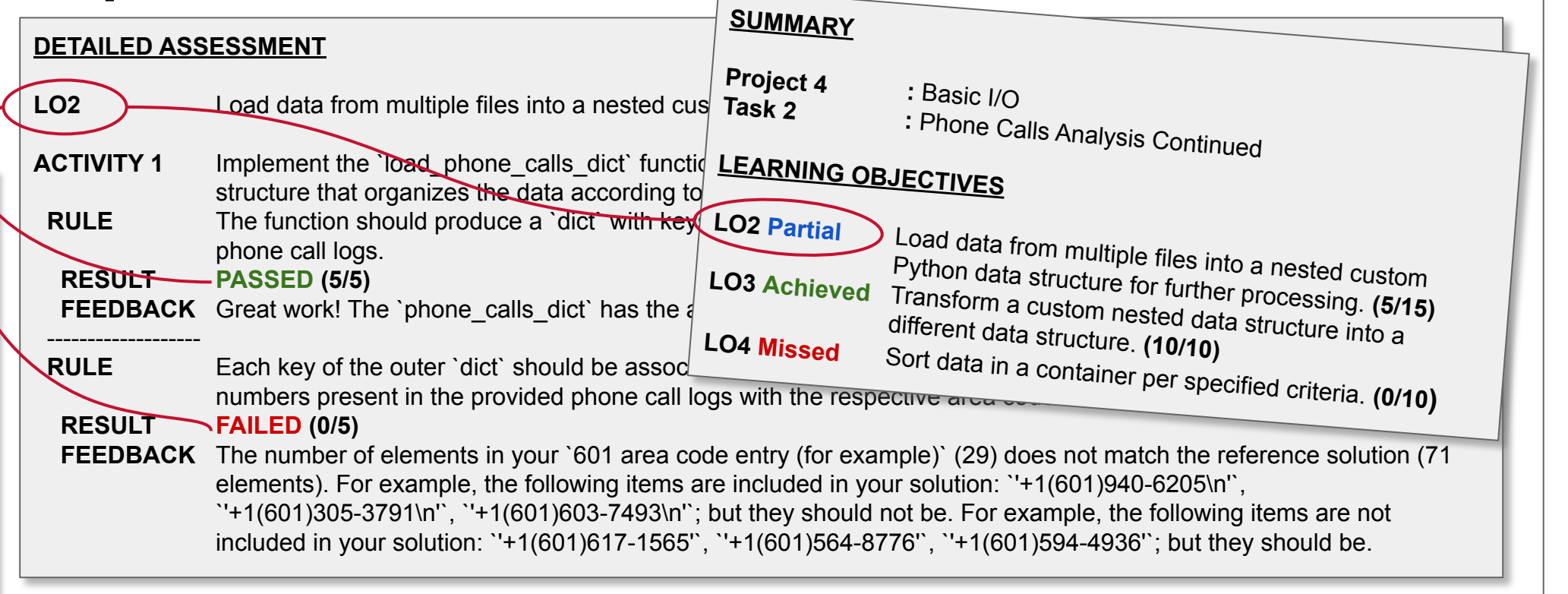
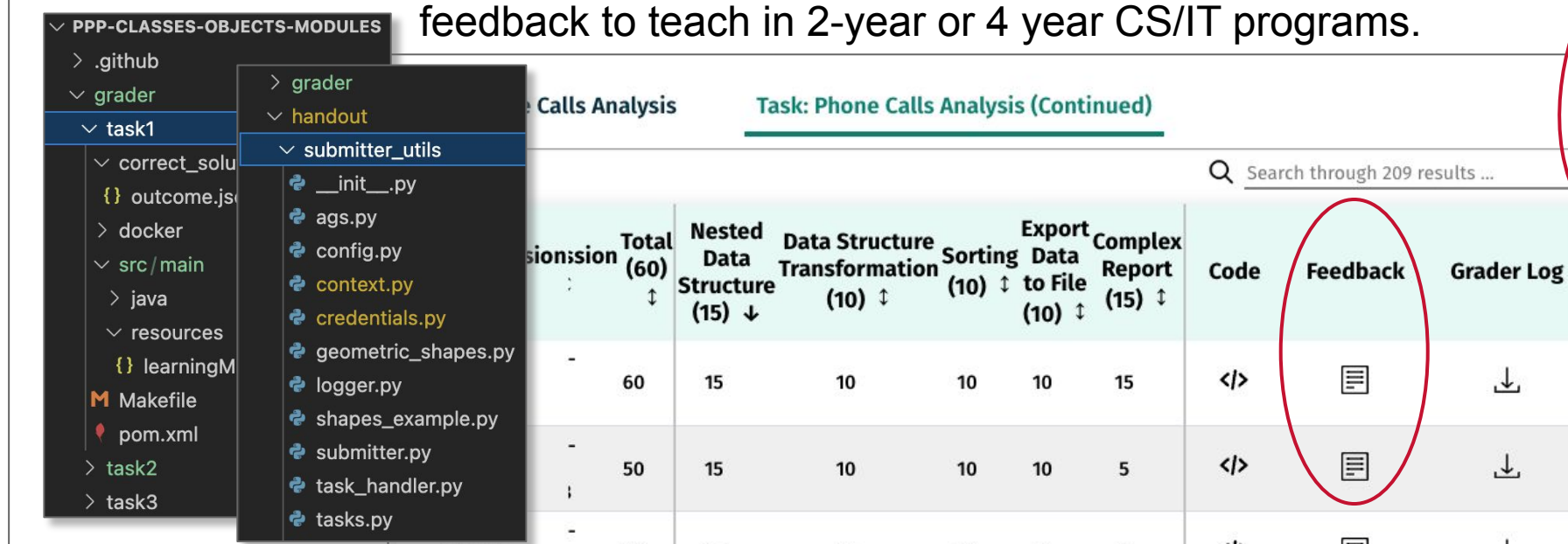
How to structure autograded CS1 project / tasks to provide flexibility for instructors' better course adaptation and for students' improved learning experience?

Course: Practical Programming with Python

	Conceptual Content	Focus of autograded projects
0	Learning with Sail()	submissions, autograder feedback
1	Python Basics and Introduction to Functions	expressions, statements, built-in and custom functions
2	Control Flow, Strings, Input and Output	if statements, repetition, relational and logical operators, string manipulation, file operations
3	Python Data Structures	insert, access, modify and delete elements in various data structures, type conversions
4	Working with Files and Modules in Python	read/write files, library use, building modules
5	Object-Oriented Programming with Python	OOP, user-defined classes and modules
6	Software Development with Python	read code, TDD, poor vs high-quality code, identify application requirements, design
7	Data Manipulation with Python	csv/tsv, json, XML, SQL, NoSQL, and related Python libraries
8	Web Scraping	HTML, CSS, web scraping, public APIs, office document processing
9	Data Analysis	pandas, data visualization, regular expressions

Automated Grading and Feedback Generation Samples from Sail()

Sail() is an online learning platform that provides technology courses created at Carnegie Mellon University that are project-based, collaborative, and use real-time feedback to teach in 2-year or 4 year CS/IT programs.



Focus Questions

1. Should all tasks be **skippable**?
 - o Which tasks can be skipped?
2. How to keep the **scope** meaningful?
3. How to keep **coherency** while skipping tasks?
4. How to **manage scores** regarding omissions?
5. What to expect as **UI changes** with modularity?

Instructors' Perspective

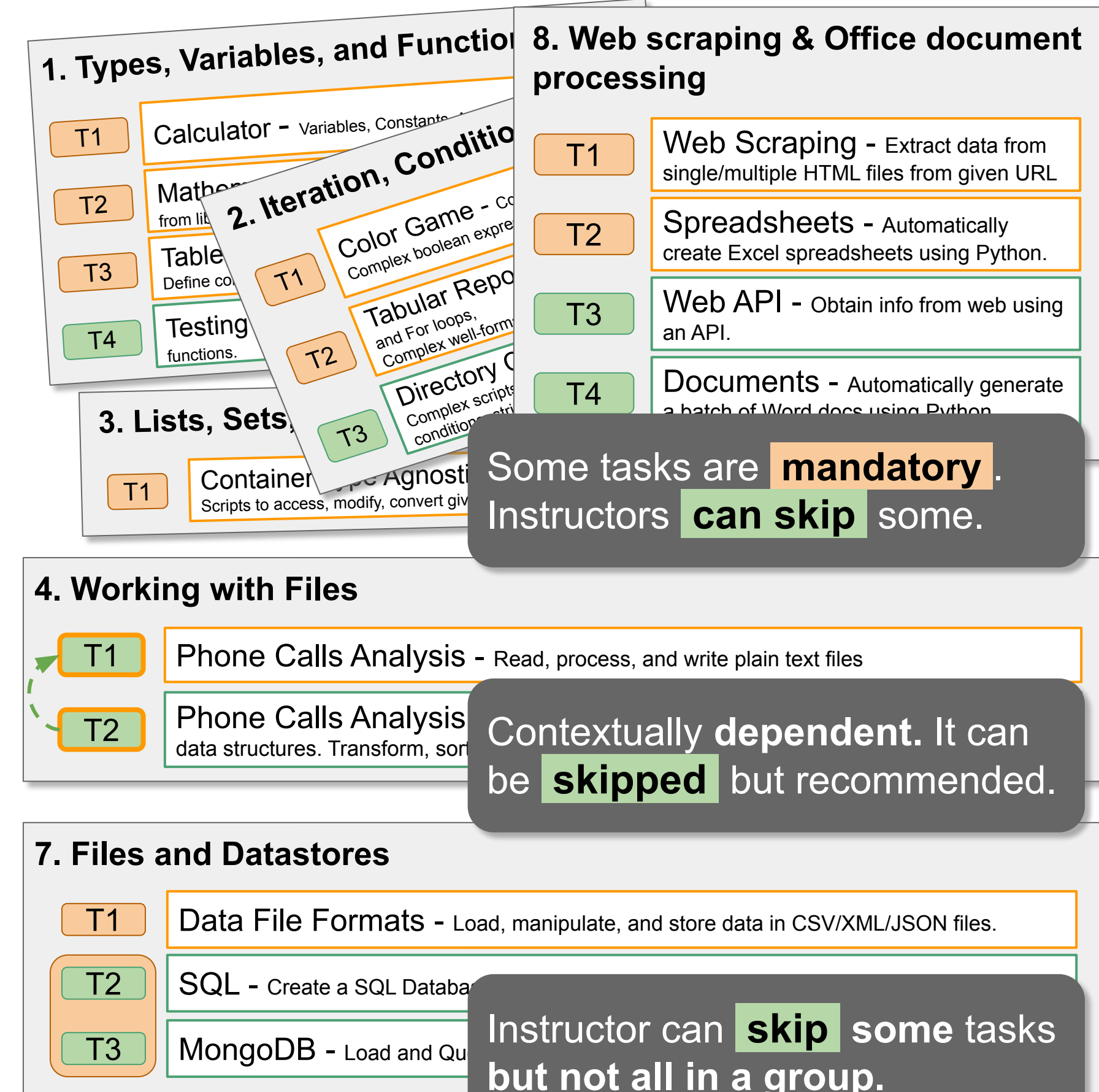
- **Adjusting** the project tasks for whole class.
 - o Skipping, hiding, replacing with alternatives
- **Overcoming the side effects** of omitting tasks
- Access to consistent **learning analytics**.

→ A flexible project-task structure would help

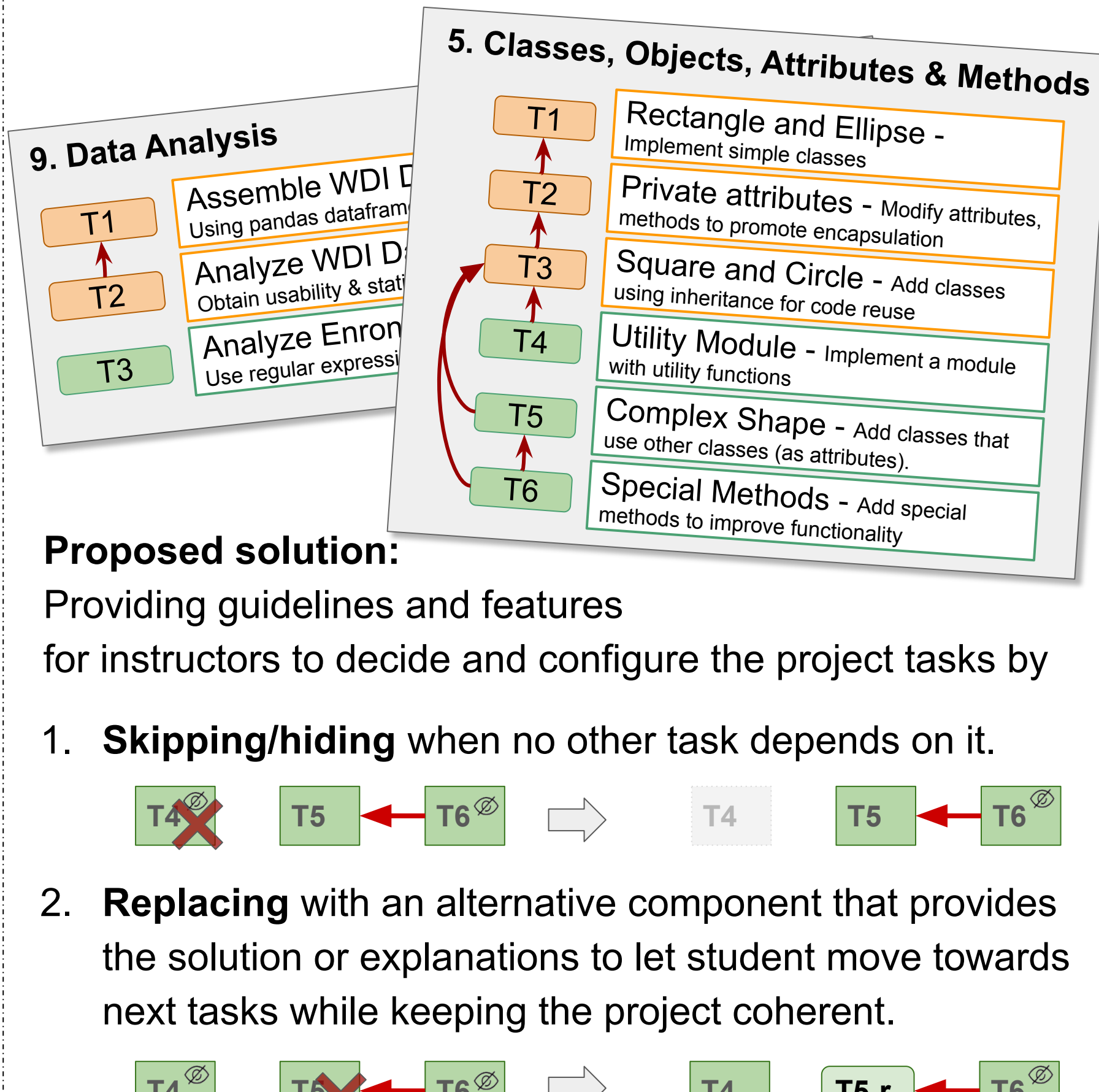
Students' Perspective

- Tasks can be highly **interdependent**.
- Students **struggling in an early task**, may not have a chance to complete the project, because of interdependency.
- Such experiences would be **discouraging** and result in **dropping out** of projects or the course.

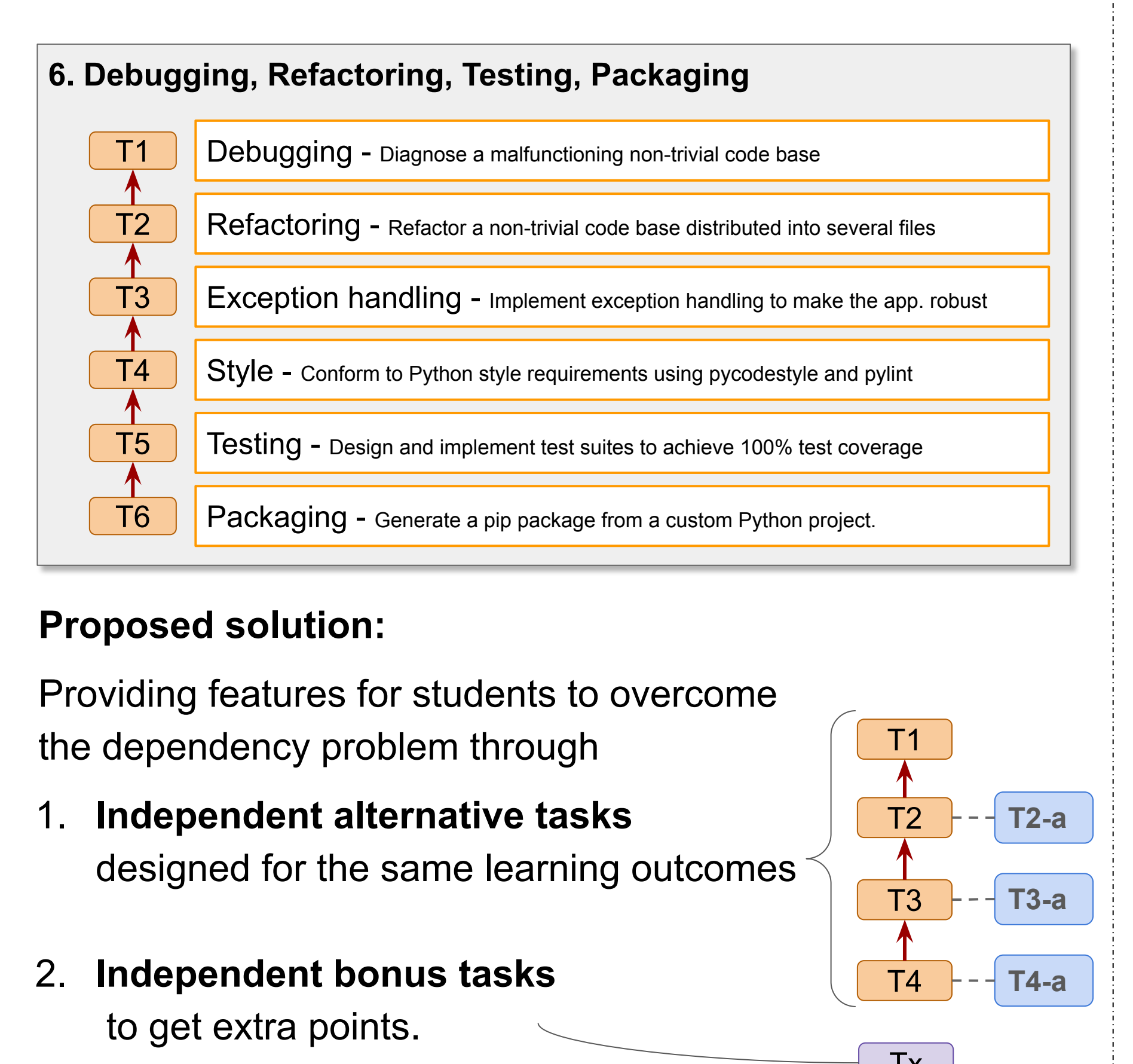
Technically independent tasks



Adding Flexibility: Optional, skippable tasks



High interdependency shouldn't block students



Handling Scores and User Interface Issues to consider

Instructors should

- easily see which tasks are **skippable** (hiding / replacing)
- **allow / hide alternative tasks** for students to have an optional path to move forward without getting stuck.
- **manage scores and weights** based on task selection.
- **manage penalty** for alternative tasks to encourage students to try the tasks with dependencies first.
- **set scores** based on difficulty relative to selected task.

Students should

- smoothly navigate tasks without getting stuck.
- be aware of the difference in scoring based on using the alternative tasks.

Project developers should consider

- learning objectives and difficulty should be same for task alternatives.
- always designing tasks with replacement and alternative tasks.

No	Task	Difficulty	Dependencies	Alternative Task
T1	Task 1 Name	1(*)	Standalone	-
T2	Task 2 Name	1.5	Prerequisite to T3	-
T3	Task 3 Name	1.5	Dependent on T2	Allow T3-A
T4	Task 4 Name	0	Dependent on T3	Allow T4-A
T5	Task 5 Name	1	Dependent on T3	Allow T5-A
T6	Task 6 Name	1	Standalone	-

Managing Modularity

e.g. T3 task focuses on inheritance using the **geometric shapes** classes created in T1 and T2

Alternate task T3-A provides **Animal and Mammal** classes and their structures. It instructs students to work on inheritance using these classes.

Instructors may skip tasks or close alternative tasks for students' access

Proposed Approach & Recommendations

- Each task should be designed for independent evaluation whenever possible.
- Tasks commonly required across many institutions/instructors should be mandatory.
- Weights of the task scores should be relative to a reference mandatory task so that scores can be handled despite task omissions.
- Alternate tasks should have less weight for scoring (being the secondary choice).

		Instructor's option	Alternative T. for students
If task is	e.g.	Hide	Replace T.
Standalone	T6	✓	✗
Dependent only	T4	✓	✗
Prerequisite only	T2	✗	✓
Dependent & Prereq.	T3	✗	✓

Task Dependencies

After adding the required alternative & replacement tasks.

